

F O U N D E R P L A Y B O O K

日课一问

A I 创 业 决 策 卡 包

100 道深度追问

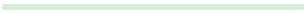
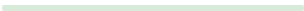
想法 · 构建 · 发布 · 增长

目录与简介

本卡包灵感来源于 Anthropic《创始人行动手册》（2026）。AI时代创业的关键转变，在于核心竞争力从「技术执行」转向「判断决策」——判断速度比执行速度更重要，对用户的深度理解比写出更快的代码更有价值。这 100 张卡片将书中的核心理念转化为可操作的每日思考框架。

卡包按创业自然流程分为四个阶段，每个阶段 25 张卡片：想法（找到真正值得解决的问题）→ 构建（用AI把想法变成产品）→ 发布（让产品找到对的用户）→ 增长（从产品到生意）。每张卡片包含一个核心问题和三个连续延伸追问，分别从个人经验、理论原则、批判反思三个维度展开，形成完整的思考链条。

阶段分类

想法	找到真正值得解决的问题	25 张	
构建	用AI把想法变成产品	25 张	
发布	让产品找到对的用户	25 张	
增长	从产品到生意	25 张	

共 100 张卡片

在AI时代，创业成功的关键到底变成了什么？当技术门槛归零，一个人能做一个团队的事，你拼的到底是什么？

你觉得现在最需要提升的能力是什么——是写代码的速度，还是做判断的质量？两者之间的差距决定了你该往哪个方向投入。当「能做出来」不再是优势，什么才是新的决定性壁垒——品味、判断力，还是对用户痛点的深刻理解？

当AI能自动化写代码、做调研、跑运营，创始人的角色应该怎么重新定义？

这周你的时间花在「亲自动手」和「指挥AI干活」的比例各是多少？编排AI agent 听起来很高大上，会不会只是换了个词说「管理团队」？当AI系统能力超过你的个人认知时，你的判断力跟得上AI的执行力吗？

技术创始人最大的优势还在吗？ 当AI让不会写代码的人也能做出 生产级软件，你的护城河还剩多 少？

一个不会写代码但很懂用户的人，和一个技术很强但不懂用户的人，谁更适合当AI时代的创始人？如果技术壁垒消失了，会不会反而让创业更难了——因为能做的人太多了？当写代码不再是核心竞争力，技术人的价值该怎么重新定义？

好想法比好技术值钱——这句话在AI时代成立吗？当想法便宜、执行力值钱，你靠什么脱颖而出？

想法满大街都是，凭什么你的想法能成？你和另外99个有同样想法的人有什么区别？未来最值钱的能力是「会用AI」还是「知道该用AI做什么」？两者的学习路径完全不同。如果一切变得可能，「不做」什么是不是比「做」更需要勇气？

你的MVP到底要验证什么？当AI让做个东西太容易，「能跑的原型」和「真正的验证」之间的区别是什么？

做完原型之后你怎么判断它是「验证成功」还是「只是做出来了」？传统创业说先做出来再说，AI时代是不是应该反过来先想清楚再说？如果验证成本极低，你会不会因为验证了太多微小的想法而错过真正需要长期投入的大机会？

AI时代的创业，什么才是真正的护城河？技术能抄、功能能抄，什么东西是竞争对手永远抄不走的？

用户对产品的依赖更多是出于使用习惯，还是因为积累了不可迁移的数据和工作流？如果技术极易被模仿，是否应该从第一天就思考数据飞轮的构建？如果技术壁垒消失了，品牌和信任会不会成为最重要的护城河？

你做的产品，自己就是用户吗？ 如果你不是目标用户，你对问题的理解到底隔了多远的一层？

你对自己产品的理解是「听说」还是「亲身经历」？如果是听说，你打算怎么弥补这层认知差距？如果你只是听说市场很大但自己从未痛过，你怎么确保做出来的东西真的能解决别人的痛？

你的创业想法，经得起反方论证吗？当AI能帮你找到无数支持你的证据，你怎么避免自己骗自己？

让AI验证想法可以找佐证，让AI压力测试想法可以找反证——你用的是哪一种模式？如果你的核心假设是错的，你能不能提前设计一个实验来证伪它，而不是等市场来打脸？有什么是你深信但大多数同行不同意的？

当做一个东西太容易，「找到问题」和「先做个东西再说」——哪种节奏更容易做出没人要的自嗨产品？

如果把「造出来」误当作「验证了」——你是否也犯过这个错？回头看，那个产品最后为什么没有成？如果AI让做东西变简单了，会不会反而更容易做出自嗨型产品，因为做的成本太低你不舍得放弃？

AI时代的创始人，最该警惕哪种旧思维？过去的成功经验，有多少已经变成了今天的毒药？

你见过哪些曾经成功的创业者却固守着完全过时的创业观念？哪些过去的成功经验，在今天的AI环境下最有可能变成毒药？先招人再做事、功能越多越好——如何系统性地识别和摆脱这些旧思维？

你的问题够不够「痛」？当用户说「这个功能不错」和「没有它我活不下去」，你听到的是同一个意思吗？

用户说「这个功能不错」和「没有它我活不下去」——你听出区别了吗？那个让你最痛、用户最急迫的功能，你把它放在优先级第一位了吗？如果你的产品明天消失了，用户的第一反应是什么？

如果你做的产品自己不用、目标用户跟自己完全不同，你怎么确保理解的需求是真实的？

你做的产品里有哪些功能是你自己经常用但用户根本不碰的？如果你不是目标用户，怎么确保自己理解的是真实需求而不是你以为的需求？当直觉和用户反馈冲突时，你通常怎么选？

你的想法够不够「小」？当AI让做什么都变得容易，你是不是反而做得太多了？

你现在的MVP有几个核心功能？如果只能留一个，你会留哪个？「少即是多」在AI时代是不是更重要了——因为用户也被信息淹没了，需要的不是更多功能而是更清晰的解决方案？

当用户说不出自己想要什么、但你的直觉告诉你应该做，你怎么判断该相信直觉还是相信沉默？

你做的产品中有哪些是「用户没想到但用了就离不开」的？这些功能是怎么被发现的？创造需求和自嗨的区别是什么——你怎么判断自己是在创新还是在臆想？

你的问题够不够「具体」？当「提高效率」和「让设计师每天少加班2小时」，你听到的是同一个问题吗？

你的产品解决的核心问题是什么？用一句话说出来，不能包含「提高」「优化」「改善」这类词。如果你的问题不够具体、不够可验证，你怎么知道你的产品在往正确的方向迭代？

当你的产品让用户越用越依赖 AI、自己动手的能力反而退化， 这算是真正的帮助吗？

你的产品是让用户变得更独立还是更依赖？如果明天你的产品消失了，用户会变得更强大还是更弱？让用户离不开和让用户变得更强大这两者矛盾吗——有没有产品同时做到了这两点？

你的想法够不够「反共识」？当所有人都说「AI会取代人类」，你有没有看到相反的可能？

当所有人都在说AI会取代人类，你有没有看到相反的可能——AI让人类腾出时间去做只有人才能做的事？你现在做的事别人没看到的机会，是真实的洞察还是孤独的偏见，你怎么验证？

当AI让做什么都变快——从想到上线只要几天——长期主义还有意义吗？

如果三年后AI能力翻十倍，你的产品还有价值吗？做产品和做项目的区别是什么——你是在解决一个长期存在的问题，还是在追一个短期热点？

你的问题够不够「真实」？当你在办公室里想象用户的痛点，和你在用户身边看到的痛点，是一回事吗？

你最近一次和目标用户面对面交流是什么时候？你看到了什么在办公室里没想到的东西？如果你不能经常见到用户，你怎么弥补这层理解上的差距？

当AI让加功能变得几乎零成本， 你怎么守住产品的简洁——加什么和砍什么的边界在哪？

用户最常用的前三个功能是什么？剩下的功能有多少是用户根本不碰的？做减法比做加法更难——你有没有勇气删掉那些你花了很多时间做的功能？

你的想法够不够「可验证」？当 你说「用户需要这个」，你有没有 办法在花钱之前证明它？

你的核心假设是什么？能不能用一句话写出来并且可以被验证？用户说他们需要和用户愿意付费——这两者有什么区别？你有没有低成本验证想法的方法？

如果你的产品让用户越用越焦虑、越用越空虚，当他们意识到的时候会怎么做？

你的产品是让用户变得更焦虑还是更平静？让用户上瘾和让用户变好——这两者矛盾吗？如果用户越用越焦虑，当他们意识到这一点时会怎么做——是卸载你，还是继续被算法困住？

你的问题够不够「紧迫」？当用户说「这个功能不错，我以后会用」，你听到的是「现在就需要」吗？

当用户说「这个功能不错，我以后会用」，你听到的是真实需求还是礼貌敷衍？以后会用和现在就需要的区别是什么——如果用户真的急需，他们会想办法现在就解决，你见过这种信号吗？

你的产品让用户用完就走、还是越用越离不开——你希望是哪一种，为什么？

你的产品是工具还是平台？工具用完就走，平台让人留下来——你有没有想过怎么让用户反复回来？如果竞争对手做出更好用的工具，用户会毫不犹豫离开你，你有没有正在搭建让他们留下的理由？

你的想法够不够「可持续」？当AI让做什么都快，你是不是反而忽略了长期价值？

如果三年后AI能力翻十倍，你的产品还有价值吗？可持续和短期热点的区别是什么——你是在解决一个长期存在的问题还是在追风口？

AI让写代码变快了，你的产品架构还需要「想清楚」吗？当重构成本极低，你是不是可以边做边改？

你最近一次因为架构问题导致返工是什么时候？如果AI让重构成本极低，那「过度设计」和「真正的架构思考」——两者的界限在哪里？你如何判断什么是必须提前想清楚的、什么可以后面再补？

当AI能帮你写出任何代码，你的核心价值在哪里——是设计系统架构，还是定义正确的问题？

你最近一次让AI帮你写代码，是你告诉它怎么写，还是它告诉你怎么写？当AI能写出任何代码，会写代码和会设计系统哪个更重要？

当AI让技术选型几乎零门槛，你的技术栈选择是基于用户需求还是个人习惯——哪种风险更大？

你选择的技术栈是因为它最适合你的产品，还是因为你最熟悉它？如果AI让你能快速切换技术栈，那选择保守还是选择激进——哪个风险更大？你有没有因为技术选型而付出过沉重的代价？

当AI让加功能变快，你怎么区分 这是在快速迭代还是在快速堆砌 ——两者之间最关键的判断标准 是什么？

你的产品有多少个功能是你加了之后后悔的？为什么当时没有拒绝？快速迭代和快速堆砌的区别是什么——你怎么判断自己是在迭代还是在堆砌？如果AI让加功能变快了，有没有配套设计「定期删功能」的机制？

你的产品是「给谁用的」？当AI让做产品变快，你是不是反而忽略了目标用户？

你的目标用户是谁？用一句话说出来，不能包含「所有人」「用户」「客户」这类词。如果你不知道目标用户是谁，你怎么知道产品做对了方向？

当AI让做出一个Demo只要几小时，从Demo到真正可上线的产品之间，你漏掉了什么？

你的产品是Demo还是产品？它有没有经过压力测试？
Demo能跑和产品能上线的区别是什么——你有没有因为忽略了工程化而返工？

当AI让功能实现越来越快，用户体验的打磨速度跟上了吗——能用和好用的差距在变大还是变小？

你的产品是「能用」还是「好用」？用户第一次用时能顺利完成核心任务吗？如果AI让做功能变快，那优化体验的优先级是否也要跟着提速？你有没有因为功能堆得快而让体验变得粗糙？

当AI能做越来越多的事，你什么事该自己做、什么事该放手——这条边界你清楚吗？

你最近一次让AI帮你做事，是你告诉它怎么做还是它告诉你怎么做？你自己做和让AI做的界限在哪里——什么事该自己做、什么事该让AI做？

当AI让加功能变得几乎没有阻力，你怎么确保产品没有偏离最初要解决的那个核心问题？

你的产品解决的核心问题是什么？用一句话说出来，不能包含任何功能名称。你做的产品中，有多少功能是直接解决核心问题的，有多少是「顺便做」的？如果AI让加功能变快，那「拒绝功能」是不是也应该变快？你有没有定期删除不相关的功能？

当验证速度越来越快，你在乎的是「学到了什么」还是「又跑完了一个实验」——两者有区别吗？

你最近一次验证失败后从中学到了什么——还是只是放弃了？快速验证和快速失败的区别是什么？如果AI让验证变快，从失败中学习的速度是不是也应该同步加速？你有没有复盘机制？

当技术实现不再是瓶颈，你怎么判断下一个功能是因为用户需要、还是因为技术上很酷？

你的产品中有多少功能是用用户要求的有多少是你觉得酷才做的？技术驱动和用户驱动的区别是什么——你有没有因为技术太强而忽略了用户需求？

当AI让你能快速交付一个又一个项目，你怎么判断自己是在积累产品资产还是在刷交付记录？

你的产品是项目还是产品？它有没有长期价值？三年后还有人用吗？「做项目」和「做产品」的区别是什么？你有没有因为追求速度而忽略了长期价值？如果AI让做项目变快，那「规划长期价值」是不是也应该变快？你有没有建立长期规划的机制？

当优化越来越快，你怎么判断什么时候该停——是用户满意了就停，还是自己满意了才算？

你的产品迭代了多少次？有多少次是必须改、有多少次只是想改？完美主义和够用就好的区别是什么——你有没有因为追求完美而错过上线时机？

当AI能帮你做很多事，找人合作的意义是变大了还是变小了——你上一次靠人而不是AI解决问题是什么时候？

你最近一次找人帮忙找的是人还是AI？自己做和找人做的界限在哪里——什么事该自己做、什么事该找人合作？如果AI能做很多事，那找人合作的意义是否反而更大了？

当功能增长没有成本约束，你怎么判断产品已经到了「功能够多」——用户的抱怨是功能太少还是太复杂？

用户最常用的前三个功能是什么？剩下的功能有多少是用户根本不碰的？功能多和体验好的区别是什么——你有没有因为功能太多而让用户体验变差？如果AI让加功能变快，删除功能的频率跟上去了吗？

当上线速度越来越快，你怎么确保质量没有下滑——你的质量门槛是在上升还是在下降？

你最近一次上线后返工的原因是什么——是需求没想清楚还是质量没把控好？快速上线和快速返工的区别是什么？如果AI让上线变快，那你在质量把控上投入的时间比例是增加了还是减少了？

当技术实现越来越容易，你怎么确保产品不是在炫技而是在解决真实痛点——用户能感受到区别吗？

你的产品中有多少功能为用户真正需要的有多少是你觉得酷才做的？技术秀和解决问题的区别是什么——你有没有因为技术太强而忽略了用户真正的问题？

当做一个工具只要几小时，你的产品凭什么让用户用完之后还想回来——它提供了什么工具不能提供的东西？

你的产品是工具还是产品？工具让用户用完就走，产品让人反复回来——你有没有想过怎么从工具进化成产品？如果竞争对手做出更好用的工具，用户会毫不犹豫离开你，你的护城河在哪里？

当开发速度越来越快，你怎么确保稳定性没有成为牺牲品——最近一次崩溃让你损失了什么？

你的产品最近一次崩溃是什么时候？崩溃的原因是代码质量还是架构问题？能用和可靠的区别是什么——你有没有因为追求速度而忽略稳定性？如果AI让开发变快，测试和监控是不是也应该跟上？

当AI让加功能快到几乎没有刹车，你怎么判断产品是在变强还是变臃肿——用户的体验变好了吗？

你的产品有多少个功能是你加了之后后悔的？快速迭代和快速堆砌的区别是什么——如果AI让加功能变快，删除功能是不是也应该变快？

当功能可以无限叠加，你怎么确保每一个新功能都在加强核心价值而不是稀释它？

你的产品提供的核心价值是什么？用一句话说出来，不能包含任何功能名称。你做的产品中，有多少功能是直接提供核心价值的，有多少是「顺便做」的？如果AI让加功能变快，那「拒绝功能」是不是也应该变快？你有没有定期删除不相关的功能？

当AI能做越来越多事，你的精力应该投在「自己做」还是「学会指挥AI」——哪个杠杆效应更大？

你最近一次让AI帮你做事，是你告诉它怎么做还是它告诉你怎么做？如果AI能帮你做很多事，那你的价值在哪里——是什么让你不可替代？

当优化可以无限进行，你怎么判断「够好」的临界点在哪里——是用户不再抱怨，还是数据不再增长？

你的产品迭代了多少次？有哪些是必须改、有哪些只是想改？完美主义和够用就好的区别是什么——你有没有因为追求完美而错过上线时机？如果AI让迭代变快，判断停下来的时机是否更难了？

当上线可以一键完成，你怎么确保每一次上线不是在制造下一次返工——上线前的检查清单够长吗？

返工的原因是什么——是需求没想清楚还是质量没把控好？快速上线和快速返工的区别是什么——你怎么判断自己是在迭代还是在返工？

当技术实现越来越炫，你的产品在用户眼里是「真厉害」还是「看不懂」——哪个评价更危险？

技术秀和解决问题的区别是什么？你有没有因为技术太强而忽略了用户真正的问题？如果AI让技术实现变快，判断技术是否值得是不是也应该变快？

你的产品做出来了，用户在哪里？当AI让做产品变快，你怎么确保产品不会变成「自嗨」？

你最近一次和目标用户交流是什么时候？你知道他们在哪里聚集、用什么工具、关注什么内容吗？如果AI让做产品变快，那找到用户是不是也应该变快？

当AI让营销内容可以批量生产， 你怎么确保自己在推广产品而不是制造信息噪音？

你最近一次推广产品，是在告诉用户产品有多好还是在告诉用户能解决什么问题？推广产品和解决问题的区别是什么——如果AI让营销变快，你有没有想过什么内容不该发？

当触达用户越来越容易，你的营销精力应该投向所有人还是精准用户——投放效率靠什么衡量？

你的目标用户是谁——用一句话说出来，不能包含「所有人」这类词。卖给所有人和卖给对的人的区别是什么？你有没有因为营销太广而浪费了资源——精准投放和广撒网，哪个效果更好？

当内容可以无限生产，你怎么判断自己在做内容还是在做营销——用户感受到的是价值还是推销？

你最近一次生产内容是为了让用户看到还是让用户理解？
做营销和做内容的区别是什么——如果AI让内容生产变快，你有没有停下来筛选什么内容真正值得生产？还是被速度推着走？

当推广可以自动化，你怎么确保每一个推广动作都在强化核心价值而不是稀释品牌？

你的产品提供的核心价值是什么？用一句话说出来，不能包含任何功能名称。你推广的内容中，有多少是直接传递核心价值的，有多少是「顺便说」的？如果AI让推广变快，那「拒绝推广」是不是也应该变快？你有没有定期删除不相关的推广内容？

当获客成本越来越低，你怎么确保获取的用户会留下来而不是漏斗里漏光——留存率在上升还是下降？

你最近一次分析用户留存是什么时候？你知道用户为什么来、为什么走吗？「获取用户」和「留住用户」的区别是什么？你有没有因为获客太强而忽略了留存？如果AI让获客变快，那「提升留存」是不是也应该变快？你有没有建立留存优化的机制？

当获客可以规模化，你怎么确保用户不是用完就走——他们第二次打开的理由是什么？

你的产品是工具还是服务？工具让用户用完就走，服务让用户持续回来——你有没有可能从工具进化成服务？如果竞争对手做出更好用的工具，你的用户会毫不犹豫离开，还是已经有留下的理由？

当推广可以自动化，你怎么确保用户是在推荐你的产品而不是你在自说自话——口碑和数据哪个更可信？

你最近一次听到用户主动推荐你的产品是什么时候？他们推荐的原因是什么？「做推广」和「做口碑」的区别是什么？你有没有因为推广太强而忽略了口碑？如果AI让推广变快，那「提升口碑」是不是也应该变快？你有没有建立口碑管理的机制？

当推广可以快速铺开，你怎么确保用户体验跟上了推广的节奏——新用户的第一印象在变好还是变差？

用户最常用的前三个功能是什么？功能多和体验好的区别是什么——你有没有因为功能太多而让体验变差？如果AI让推广变快，你优化体验的速度是否跟上了推广吸引来的用户预期？

当营销可以大规模复制，你的品牌是在积累长期信任还是在透支短期注意力——用户还记得你吗？

你最近一次做品牌活动是什么时候？你知道品牌的核心价值是什么吗？「做营销」和「做品牌」的区别是什么？你有没有因为营销太强而忽略了品牌？如果AI让营销变快，那「建设品牌」是不是也应该变快？你有没有建立品牌管理的机制？

当触达可以无限扩张，你的目标用户越精准还是越模糊——上一次精准找到用户是什么时候？

你的目标用户是谁——用一句话说出来，不能包含「所有人」这类词。你有没有因为营销太广而浪费了资源？精准找到对的用户和广撒网——哪个更适合你现在产品所处的阶段？

当内容生产没有上限，你怎么确保每一条内容都在传递价值而不是填充发布频率——用户打开率在变吗？

你最近一次生产内容是为了让用户看到还是让用户理解？
做推广和做内容的区别是什么？如果AI让内容生产变快，
判断什么内容值得生产是不是也应该变快？

当推广可以无限叠加，你怎么确保每一个推广动作都在服务核心价值——用户是被吸引来的还是被喊来的？

你的产品提供的核心价值是什么？用一句话说出来，不能包含任何功能名称。你推广的内容中，有多少是直接传递核心价值的，有多少是「顺便说」的？如果AI让推广变快，那「拒绝推广」是不是也应该变快？你有没有定期删除不相关的推广内容？

当获客渠道越来越多，你的精力是花在新客获取还是老客留存——哪个对当下的你更重要？

你最近一次分析用户留存是什么时候？你知道用户为什么来、为什么走吗？「获取用户」和「留住用户」的区别是什么？你有没有因为获客太强而忽略了留存？如果AI让获客变快，那「提升留存」是不是也应该变快？你有没有建立留存优化的机制？

当用户可以无限获取，你怎么确保他们不是用完一次就再也不回来——你的产品留下了什么让他们记住？

你的产品是工具还是服务？工具用完就走，服务让人持续回来——你有没有想过怎么让用户反复使用而不是一次就弃？如果竞争对手做出更好用的工具，你的用户是否有不走的理由？

当推广可以自动化，你怎么确保用户愿意主动推荐你的产品——上一次用户主动推荐是什么时候？

你最近一次听到用户主动推荐你的产品是什么时候？他们推荐的原因是什么？「做营销」和「做口碑」的区别是什么？你有没有因为营销太强而忽略了口碑？如果AI让营销变快，那「提升口碑」是不是也应该变快？你有没有建立口碑管理的机制？

当功能可以无限堆叠，你的用户是在享受更多功能还是在忍受更多复杂度——你最想删掉哪个功能？

用户最常用的前三个功能是什么？剩下的功能有多少是用户根本不碰的？功能多和体验好的区别是什么——你有没有因为功能太多而让用户体验变差？砍掉最不用的那个功能你敢吗？

当营销可以持续轰炸，你的品牌在用户心里是越来越清晰还是越来越模糊——一句话能说清楚你是谁吗？

你最近一次做品牌活动是什么时候？你知道品牌的核心价值是什么吗？「做营销」和「做品牌」的区别是什么？你有没有因为营销太强而忽略了品牌？如果AI让营销变快，那「建设品牌」是不是也应该变快？你有没有建立品牌管理的机制？

当触达可以无限精准，你的目标用户画像是在变清晰还是变模糊——用一个场景描述他们，你能吗？

你的目标用户是谁——用一句话说出来。你有没有因为营销太广而浪费了资源？精准找到对的用户和广撒网——哪个更适合你现在产品所处的阶段？上一次精准触达用户是什么时候？

当内容可以无限生产，你怎么确保自己不是在制造信息垃圾——用户看完内容后的下一步是什么？

你最近一次生产内容是为了让用户看到还是让用户理解？
做推广和做内容的区别是什么——如果AI让内容生产变快，你有没有停下来筛选什么真正值得生产？还是被速度推着走？

当推广可以自动化，你怎么确保产品没有在推广中偏离了核心价值——新用户对你的第一印象准确吗？

你的产品提供的核心价值是什么？用一句话说出来，不能包含任何功能名称。你推广的内容中，有多少是直接传递核心价值的，有多少是「顺便说」的？如果AI让推广变快，那「拒绝推广」是不是也应该变快？你有没有定期删除不相关的推广内容？

当获客可以批量完成，你的用户质量是在提升还是下降——最近一批用户的生命周期数据说了什么？

你最近一次分析用户留存是什么时候？你知道用户为什么来、为什么走吗？「获取用户」和「留住用户」的区别是什么？你有没有因为获客太强而忽略了留存？如果AI让获客变快，那「提升留存」是不是也应该变快？你有没有建立留存优化的机制？

当获客规模可以无限扩大，你怎么确保用户不是一锤子买卖——复购率和续费率在哪个方向走？

你的产品是工具还是服务？一次性和持续性的区别是什么——你有没有可能从工具进化成服务？如果竞争对手做出更好的工具，用户会离开你吗？

当推广可以自动化，你的产品是靠广告活着还是靠口碑活着——关掉所有广告还能活多久？

你最近一次听到用户主动推荐你的产品是什么时候？他们推荐的原因是什么？「做营销」和「做口碑」的区别是什么？你有没有因为营销太强而忽略了口碑？如果AI让营销变快，那「提升口碑」是不是也应该变快？你有没有建立口碑管理的机制？

当功能可以无限添加，你的用户是在感谢更多选择还是在抱怨更难上手——新用户的激活率在上升还是下降？

用户最常用的前三个功能是什么？剩下的功能有多少用户根本不碰？功能多和体验好的区别是什么——你有没有因为功能太多而让用户体验变差？砍掉最不用的那个功能你敢吗？

你的产品能赚钱吗？当AI让做产品变快，你怎么确保产品不会变成「叫好不叫座」？

你知道每个用户的获取成本是多少、生命周期价值是多少吗？有人用和能赚钱的区别是什么——你有没有因为忽略了商业模式而让产品无法持续？如果明天所有广告停了，你的产品还能活下去吗？

当AI让做产品变成一件极低成本的事，你怎么确保产品不是「能用的Demo」而是「能赚钱的生意」？

你知道产品的毛利率、净利率、现金流吗？做产品和做生意的区别是什么——你有没有因为忽略了财务而让产品无法持续？如果明天投资人撤资，你自己能不能撑下去？

你的产品能规模化吗？当AI让做产品变快，你怎么确保产品不会变成「小而美」？

你知道产品的边际成本和扩展成本吗？做得好和做得大的区别是什么——你有没有因为忽略了规模化路径而让增长停滞？如果用户量突然翻十倍，你的系统撑得住吗？

当AI能帮你做越来越多事，找人合作的意义是变大了还是变小了——你上一次把一件事完全交给别人是什么时候？

你最近一次找人帮忙找的是人还是AI？自己做和找人做的界限在哪里——什么事该自己做、什么事该找人合作？如果AI能做很多事，那找人合作的意义是否反而更大了？

当功能可以无限添加，你怎么确保每一个新功能都在强化产品最核心的那个价值主张？

你的产品提供的核心价值是什么？用一句话说出来，不能包含任何功能名称。你做的产品中，有多少功能是直接提供核心价值的，有多少是「顺便做」的？如果AI让加功能变快，那「拒绝功能」是不是也应该变快？你有没有定期删除不相关的功能？

当AI让加功能快到没有刹车，你怎么确保产品没有变成一坨臃肿的代码——最近删掉的一个功能是什么？

你的产品有多少个功能是你加了之后后悔的？如果AI让加功能变快，删除功能是不是也应该变快——你有没有定期做减法的习惯？上一个被你删掉的功能是什么？

当技术实现越来越炫，你怎么确保用户不是在喊「酷」而是在说「有用」——两者的区别你上次什么时候感受到的？

技术秀和解决问题的区别是什么——你有没有因为技术太强而忽略了用户真正的问题？如果AI让技术实现变快，判断技术是否值得投入是不是也应该更快做出？上一个被你放弃的技术决策是什么？

当AI让你能快速交付一个又一个项目，你怎么判断哪一个值得长期投入——判断标准是什么？

你的产品是项目还是产品？它有没有长期价值？三年后还有人用吗？「做项目」和「做产品」的区别是什么？你有没有因为追求速度而忽略了长期价值？如果AI让做项目变快，那「规划长期价值」是不是也应该变快？你有没有建立长期规划的机制？

当优化可以无限迭代，你怎么判断「够好」的临界点——是用户说好、数据说好、还是你自己说好？

你的产品迭代了多少次？完美主义和够用就好的区别是什么——你有没有因为追求完美而错过上线时机？如果AI让迭代变快，判断什么时候该停止是不是反而更难了？

当上线速度越来越快，你怎么确保自己没有制造下一次返工——上线前你必查的三件事是什么？

返工的原因是什么——是需求没想清楚还是质量没把控好？快速上线和快速返工的区别是什么——你怎么判断自己是在迭代还是在不断返工？上一次返工的教训你用到了下一次吗？

当开发越来越快，你怎么确保可靠性没有成为牺牲品——最近一次崩溃的根因是什么？

你的产品最近一次崩溃是什么时候？崩溃的原因是什么——能用和可靠的区别你有没有认真想过？如果AI让开发变快，你测试和监控的投入比例是增加了还是减少了？

当AI能做越来越多事，你的精力应该投在哪个方向——是自己更精通一个领域，还是学会高效指挥AI？

你最近一次让AI帮你做事，是你告诉它怎么做还是它告诉你怎么做？如果AI能帮你做很多事，你的价值在哪里——是什么让你不可替代？

当功能可以无限堆叠，你的用户最常用的功能和最不常用的功能之间差距有多大——你敢砍掉最不常用的那个吗？

用户最常用的前三个功能是什么？功能多和体验好的区别是什么——你有没有因为功能太多而让用户体验变差？砍掉最不用的那个功能你敢吗？

当AI让加功能快到没有阻力，你怎么确保产品没有变成功能过剩、体验不足的泥潭？

你的产品有多少个功能是你加了之后后悔的？快速迭代和快速堆砌的区别是什么——如果AI让加功能变快，删除功能是不是也应该变快？

当技术实现越来越炫，你的产品离用户真正的问题是越来越近还是越来越远——用户自己怎么说的？

技术秀和解决问题的区别是什么——你有没有因为技术太强而忽略了用户真正的问题？如果AI让技术实现变快，判断技术是否值得投入是不是也应该更快做出？上一次你放弃的是什么技术？

当AI让你能快速交付一个又一个成果，你怎么判断自己是在盖房子还是在摆积木——哪个能经得起三年考验？

你的产品是项目还是产品？它有没有长期价值？三年后还有人用吗？「做项目」和「做产品」的区别是什么？你有没有因为追求速度而忽略了长期价值？如果AI让做项目变快，那「规划长期价值」是不是也应该变快？你有没有建立长期规划的机制？

当优化可以无限进行，你怎么判断「到此为止」的时机——是用户不再抱怨，还是你已经不再在乎？

你的产品迭代了多少次？完美主义和够用就好的区别是什么——如果AI让优化变快，判断什么时候该停止是不是反而更难了？上一次你因为追求完美而延误的事情，现在回头看后悔吗？

当上线可以一键完成，你怎么确保每一次上线都在减少而非增加技术债——你的代码质量在变好还是变差？

返工的原因是什么——是需求没想清楚还是质量没把控好？快速上线和快速返工的区别是什么——如果AI让上线变快，你的质量门槛是提高了还是降低了？

当开发速度越来越快，你怎么确保可用性没有成为牺牲品——最近一次宕机让你损失了什么？

你的产品最近一次崩溃是什么时候？崩溃的原因是什么——能用和可靠的区别你有没有认真想过？如果AI让开发变快，你投入测试和监控的时间比例是增加了还是减少了？

当AI能做越来越多事，你的价值在哪里——是你会用AI，还是你知道该让AI做什么？

你最近一次让AI帮你做事，是你告诉它怎么做还是它告诉你怎么做？如果AI能帮你做很多事，你的不可替代性在哪里——是判断力、是品味、还是对用户的深刻理解？

当功能可以无限添加，你的用户是在享受便利还是在忍受复杂——少一个功能他们会更开心吗？

用户最常用的前三个功能是什么？剩下的功能有多少用户根本不碰？功能多和体验好的区别是什么——如果AI让加功能变快，你有没有配套设计删除功能的节奏？

当AI让加功能快到没有刹车，你怎么确保产品在变强而不是变胖——你有定期做「减脂」吗？

你的产品有多少个功能是你加了之后后悔的？为什么当时没有拒绝？如果AI让加功能变快，删除功能是不是也应该变快——你上一次删掉一个功能是什么时候？

当技术实现越来越容易，你的产品是在解决真实问题还是在满足技术好奇心——用户买了单吗？

技术秀和解决问题的区别是什么？你有没有因为技术太强而忽略了用户真正的问题？如果AI让技术实现变快，判断技术是否值得是不是也该变快？

当AI让你能快速交付一个又一个成果，你怎么判断自己是在做产品还是在刷履历——三年后回头看你会骄傲吗？

你的产品是项目还是产品？它有没有长期价值？三年后还有人用吗？「做项目」和「做产品」的区别是什么？你有没有因为追求速度而忽略了长期价值？如果AI让做项目变快，那「规划长期价值」是不是也应该变快？你有没有建立长期规划的机制？

你的创业成功到底靠实力还是运气？当运气无处不在，你怎么知道哪些是你复制的？

回想你最成功的一次决策，事后看有多少因素是你完全可控的？如果你承认运气占了很大比重，那你是不是应该区分清楚——哪些是可复制的，哪些只是碰巧对了？运气成为常态时，可复制的成功和一次性的爆款区别在哪里？